

```

import optuna
import numpy as np
from typing import Tuple, Optional, Callable
import time
from dataclasses import dataclass

## =====
## 1. Data Structures
## =====

@dataclass
class MPOState:
    """Tracks the state of MPO execution"""
    executed_volume: float = 0.0
    cumulative_pnl: float = 0.0
    current_step: int = 0
    active_orders: list = None
    last_print: float = 0.0

## =====
## 2. Core Simulation
## =====

class MPOSimulator:
    def __init__(self, config: dict):
        """
        Initialize MPO simulator with market parameters

        Args:
            config: Dictionary containing:
                - target_volume: Total volume target
                - time_steps: Number of simulation steps
                - market_volatility: Price movement volatility
                - base_liquidity: Base market liquidity
                - tick_size: Minimum price increment
        """
        self.config = config
        self.reset()

    def reset(self):
        """Reset simulation state"""
        self.state = MPOState()
        self.state.active_orders = []
        self.reference_price = 100.0 # Starting price

```

```

def run(
    self,
    aggression: float,
    spread: float,
    volume_callback: Optional[Callable] = None,
    verbose: bool = False
) -> Tuple[float, float]:
    """
    Execute MPO simulation with given parameters

    Args:
        aggression: Order placement aggression (0.1-1.0)
        spread: Order spread from mid (0.5-2.0)
        volume_callback: Function for volume checks
        verbose: Print progress if True

    Returns:
        tuple: (final_pnl, executed_volume)
    """
    self.reset()
    start_time = time.time()

    for step in range(self.config['time_steps']):
        self.state.current_step = step

        # 1. Generate market movement
        self._update_market_state()

        # 2. Determine order placement
        self._place_orders(aggression, spread)

        # 3. Simulate order execution
        self._execute_orders()

        # 4. Check volume thresholds
        if volume_callback:
            volume_callback(self.state.executed_volume)

        # 5. Early termination if target reached
        if self.state.executed_volume >= self.config['target_volume']:
            break

        # Optional progress reporting
        if verbose and time.time() - self.state.last_print > 1.0:
            print(f"Step {step}: {self.state.executed_volume:.0f}/{self.config['target_volume']}")

```

```

        self.state.last_print = time.time()

    # Calculate final metrics
    execution_time = time.time() - start_time
    slippage = self._calculate_slippage(aggression)
    final_pnl = self.state.cumulative_pnl - slippage

    if verbose:
        print(f"Completed in {execution_time:.2f}s | PnL: {final_pnl:.2f} | Volume: {self.state.executed_volume:.0f}")

    return final_pnl, self.state.executed_volume

def _update_market_state(self):
    """Simulate market price movement"""
    price_move = np.random.normal(0, self.config['market_volatility'])
    self.reference_price *= (1 + price_move)

def _place_orders(self, aggression: float, spread: float):
    """Generate orders based on current state"""
    remaining_volume = self.config['target_volume'] - self.state.executed_volume
    available_liquidity = max(
        self.config['base_liquidity'] * 0.1,
        self.config['base_liquidity'] * (1 - aggression)
    )

    # Calculate order size
    order_size = min(
        remaining_volume,
        available_liquidity * aggression
    )

    if order_size > 0:
        # Simple limit order placement
        order_price = self.reference_price * (1 + (spread * 0.0001))
        self.state.active_orders.append({
            'price': order_price,
            'size': order_size,
            'filled': 0.0
        })

def _execute_orders(self):
    """Simulate order fills with partial execution"""
    for order in self.state.active_orders:
        # Random fill ratio between 0-100%
        fill_ratio = np.random.uniform(0, 1)
        filled = min(order['size'] - order['filled'], order['size'] * fill_ratio)

```

```

        # Update state
        order['filled'] += filled
        self.state.executed_volume += filled
        self.state.cumulative_pnl += filled * (self.reference_price - order['price'])

    # Remove fully filled orders
    self.state.active_orders = [
        o for o in self.state.active_orders
        if o['filled'] < o['size']
    ]

def _calculate_slippage(self, aggression: float) -> float:
    """Estimate market impact costs"""
    return 0.01 * aggression * (self.state.executed_volume ** 1.5)

## =====
## 3. Optuna Optimization
## =====

class VolumePruner:
    def __init__(self, min_volume: float, warmup_steps: int = 5):
        """
        Args:
            min_volume: Minimum volume threshold
            warmup_steps: Number of steps before checking volume
        """
        self.min_volume = min_volume
        self.warmup_steps = warmup_steps

    def __call__(self, trial: optuna.Trial, current_step: int, current_volume: float):
        """Check volume and prune if below threshold"""
        if current_step > self.warmup_steps and current_volume < self.min_volume:
            trial.set_user_attr("prune_reason", f"Volume {current_volume:.0f} < {self.min_volume} at step {current_step}")
            raise optuna.TrialPruned()

def create_study(config: dict) -> optuna.Study:
    """Configure Optuna study with custom settings"""
    sampler = optuna.samplers.TPESampler(
        consider_prior=True,
        prior_weight=1.0,
        consider_magic_clip=True,
        consider_endpoints=False,
        n_startup_trials=20
    )

```

```

pruner = optuna.pruners.MedianPruner(
    n_startup_trials=10,
    n_warmup_steps=5,
    interval_steps=2
)

return optuna.create_study(
    directions=["maximize", "maximize"], # PnL and volume
    sampler=sampler,
    pruner=pruner,
    study_name="MPO_Optimization"
)

def objective(trial: optuna.Trial, config: dict) -> Tuple[float, float]:
    """Optuna objective function with constraints"""
    # 1. Initialize components
    simulator = MPOSimulator(config)
    volume_pruner = VolumePruner(
        min_volume=config['min_volume'],
        warmup_steps=config['warmup_steps']
    )

    # 2. Suggest parameters
    params = {
        'aggression': trial.suggest_float("aggression", 0.1, 1.0, step=0.05),
        'spread': trial.suggest_float("spread", 0.5, 2.0, step=0.1),
        'rebalance_freq': trial.suggest_int("rebalance_freq", 1, 10)
    }

    # 3. Run simulation
    try:
        pnl, volume = simulator.run(
            aggression=params['aggression'],
            spread=params['spread'],
            volume_callback=lambda vol: volume_pruner(trial, simulator.state.current_step,
vol)
        )
    except optuna.TrialPruned:
        return float('-inf'), float('-inf')

    # 4. Apply constraints
    if pnl <= 0 or volume < config['min_volume']:
        trial.set_user_attr("constraint_violation", True)
        return float('-inf'), float('-inf')

    # 5. Store additional metrics
    trial.set_user_attr("execution_time", simulator.state.current_step)

```

```

trial.set_user_attr("slippage", simulator._calculate_slippage(params['aggression']))

return pnl, volume

## =====
## 4. Execution & Analysis
## =====

def optimize_mpo(config: dict, n_trials: int = 100) -> optuna.Study:
    """Run complete optimization process"""
    study = create_study(config)

    # Add known good parameters as initial trials
    study.enqueue_trial({
        'aggression': 0.5,
        'spread': 1.0,
        'rebalance_freq': 5
    })

    # Run optimization
    study.optimize(
        lambda trial: objective(trial, config),
        n_trials=n_trials,
        timeout=3600, # 1 hour timeout
        show_progress_bar=True
    )

    return study

def analyze_results(study: optuna.Study):
    """Analyze and visualize optimization results"""
    print(f"Number of completed trials: {len(study.trials)}")
    print(f"Number of pruned trials: {len([t for t in study.trials if t.state ==
optuna.trial.TrialState.PRUNED])}")

    # Get best trial (Pareto front for multi-objective)
    best_trials = study.best_trials
    for i, trial in enumerate(best_trials):
        print(f"\nBest trial #{i+1}:")
        print(f" Params: {trial.params}")
        print(f" Values: PnL={trial.values[0]:.2f}, Volume={trial.values[1]:.0f}")
        print(f" Duration: {trial.user_attrs.get('execution_time', 'N/A')} steps")

    # Plot optimization history
    fig = optuna.visualization.plot_optimization_history(study)
    fig.show()

```

```
## =====
## 5. Main Execution
## =====

if __name__ == "__main__":
    # Configuration
    config = {
        'target_volume': 10000,
        'min_volume': 2000, # Early pruning threshold
        'time_steps': 100,
        'market_volatility': 0.001,
        'base_liquidity': 5000,
        'tick_size': 0.01,
        'warmup_steps': 5
    }

    # Run optimization
    study = optimize_mpo(config, n_trials=100)

    # Analyze results
    analyze_results(study)
```